



Real-Time Edge Deepfake Detector: Master Workflow & Architecture

Since your team is building this from scratch, the project must be split into two main tracks that can be worked on in parallel: **Track A (Machine Learning & Data)** and **Track B (Client Applications & Edge Optimization)**.

1. Required Tech Stack & Programming Languages

To build this end-to-end, your team will need proficiency in the following:

- **Python:** The undisputed king of Machine Learning. Used for downloading datasets, preprocessing audio/video, defining neural networks, and training the models. (Libraries: PyTorch or TensorFlow, OpenCV, Librosa, HuggingFace).
- **TypeScript / JavaScript:** Required for the Browser Extension, DOM manipulation, and handling WebGL/WebGPU and Web Workers.
- **C++ or Rust (Optional but Recommended):** If you are building a *Desktop Daemon* that intercepts system-level audio/video (outside the browser), you need a low-level systems language for performance and OS API hooks.
- **HTML/CSS:** For the user interface and warning overlays.
- **ONNX (Open Neural Network Exchange):** The format you will use to bridge your Python-trained models with your JavaScript/C++ edge clients.

2. Phase 1: Data Acquisition & Preprocessing (Python)

The models are only as good as the data they learn from.

The Goal: Gather massive datasets of real and fake media and process them into a format the neural networks can digest.

- **Video Datasets:** Your team needs to download standard deepfake datasets like *FaceForensics++*, *Celeb-DF*, or the *Deepfake Detection Challenge (DFDC)* dataset.
- **Audio Datasets:** Download datasets like *ASVspoof* or the *FakeAVCeleb* dataset which contain authentic and AI-generated voices.
- **Video Preprocessing Pipeline:**
 1. Extract frames from videos.
 2. Use a fast face-detection algorithm (like MediaPipe or MTCNN) to crop exactly to the subject's face.
 3. Normalize the colors and resize all face crops to a standard size (e.g., 224x224 pixels).
- **Audio Preprocessing Pipeline:**
 1. Convert audio clips to a standard sample rate (e.g., 16kHz).
 2. Convert the raw audio waves into visual representations called *Mel-spectrograms*. Deep learning models are surprisingly good at treating audio as an image and looking for visual anomalies in the frequency phases.

3. Phase 2: Model Architecture & Training (Python)

Building the brain of the detector.

The Goal: Train two separate, highly optimized neural networks. One for video, one for audio.

- **The Video Model (Spatial & Temporal):**
 - You need a model that looks at single frames for artifacts (Spatial) and looks across multiple frames for unnatural movement, like weird blinking (Temporal).
 - *Architecture:* Start with an EfficientNet (for lightweight image feature extraction) combined with a small LSTM (Long Short-Term Memory network) or a Vision Transformer (ViT) to track changes over time.
 - *Training:* Train the model to output a binary classification: 0 (Real) or 1 (Fake).
- **The Audio Model:**
 - *Architecture:* A Convolutional Neural Network (CNN) that analyzes the Mel-spectrograms you generated. Synthetic voices often lack high-frequency breathing sounds or have robotic phase alignments that the CNN can learn to spot.
- **The Constraint:** Because this must run on the edge, you cannot use massive 1-Billion parameter models. You must strictly limit the model size (target under 20-50MB per model).

4. Phase 3: Edge Optimization & Export

Making the heavy models lightweight.

The Goal: Python PyTorch models cannot run natively in a browser. They must be converted and compressed.

1. **Export to ONNX:** Convert your trained PyTorch/TensorFlow models into the `.onnx` graph format.
2. **Quantization:** This is crucial. Neural networks usually use 32-bit floating-point numbers (FP32). You will run a quantization script to convert these to 16-bit (FP16) or even 8-bit integers (INT8). This slashes the memory size of your model by 50% to 75% with minimal loss in accuracy, making it viable for browser execution.

5. Phase 4: Building the Browser Extension (TypeScript)

The deployment vehicle for the web.

The Goal: Intercept media, run the ONNX model, and show warnings.

1. **The Manifest & Background Worker:** Set up a Manifest V3 extension. The background script manages permissions and orchestrates the extension.
2. **Stream Interception:** * Inject a Content Script into the active webpage.
 - Find the `<video>` or `<audio>` elements playing on the page.
3. **The Web Worker (Crucial):** *Do not run ML models on the main UI thread.* It will freeze the user's browser. Spawn a Web Worker (a background JavaScript thread).
4. **The Inference Loop:**
 - Draw the playing video to an off-screen HTML Canvas at a set interval (e.g., 3 frames per second).
 - Extract the pixel data and send it to the Web Worker.
 - Inside the Web Worker, use `onnxruntime-web` to process the pixels through your quantized ONNX model using the user's GPU (via WebGPU or WebGL).
5. **Alerting:** If the Web Worker returns a high "Fake" probability, the Content Script injects a red warning UI over the video player.

6. Phase 5: Building the Desktop Daemon (Rust/C++) - *Optional but powerful*

For detecting deepfakes in apps like Zoom, Teams, or Discord.

The Goal: Intercept media at the OS level before it reaches the communication app.

1. **Virtual Audio Cable / Virtual Camera:** Create a virtual device driver on Windows/macOS.
2. **OS Hooks:** Use OS APIs to capture the system's screen buffer or audio output buffer.
3. **Native Inference:** Use the C++ ONNX Runtime natively on the machine to process the buffers.
4. **System Overlay:** Use OS-level windowing APIs to draw an always-on-top, click-through warning overlay on the user's screen if a deepfake is detected on a Zoom call.

7. Phase 6: Smoothing & False Positive Reduction

Making it feel like magic, not a glitchy prototype.

A single frame might be falsely flagged due to poor internet connection or video compression (like a blocky YouTube video).

- **Rolling Average:** Implement a queue that stores the last 15 predictions.
- Only trigger the "Deepfake Detected" warning if, say, 80% of the last 15 predictions returned a high fake score. This prevents the warning UI from rapidly flickering on and off.